Politechnika Wrocławska

Identyfikacja i modelowanie statystyczne

Projekt

Identyfikacja nieliniowych systemów dynamicznych
z wykorzystaniem rekurencyjnych sieci neuronowych

Autorzy:

X
X

Wrocław, 2026

Spis treści

1 Wstęp **2**

2 Badane układy dynamiczne **2**
2.1 Oscylator Van der Pola 2
2.2 Oscylator Duffinga 3

3 Generacja danych treningowych **3**
3.1 Numeryczne całkowanie ODE 3
3.2 Parametry zestawu danych 3
3.3 Metoda okna przesuwnego 4

4 Architektury sieci rekurencyjnych **4**
4.1 Komórka LSTM 4
4.2 Komórka GRU 5
4.3 Wspólna architektura predyktora 5

5 Procedura treningu **6**
5.1 Funkcja straty i optymalizator 6
5.2 Harmonogram współczynnika uczenia 6
5.3 Wczesne zatrzymanie 6
5.4 Przycinanie gradientów 6
5.5 Środowisko obliczeniowe 6

6 Wyniki **7**
6.1 Tabela porównawcza metryk 7
6.2 Krzywe uczenia 7
6.3 Predykcja wielokrokowa 8
6.4 Narastanie błędu predykcji 9

7 Dyskusja wyników **10**
7.1 Oscylator Van der Pola 10
7.2 Oscylator Duffinga 10
7.3 Wpływ chaosu na horyzont predykcji 10
7.4 Czas treningu 11

8 Podsumowanie i dalsze kroki **11**

1 Wstęp

Identyfikacja systemów dynamicznych jest fundamentalnym zagadnieniem w teorii sterowania, fizyce oraz inżynierii. Celem identyfikacji jest zbudowanie modelu matematycznego układu na podstawie obserwowanych danych wejściowo-wyjściowych, bez pełnej znajomości praw rządzących danym układem.

Tradycyjne metody identyfikacji – oparte na modelach liniowych, rozwinięciach szeregowych lub metodzie najmniejszych kwadratów – zawodzą w przypadku układów silnie nieliniowych lub chaotycznych. W takich scenariuszach atrakcyjną alternatywę stanowią **rekurencyjne sieci neuronowe** (ang. *Recurrent Neural Networks*, RNN), które potrafią aproksymować dowolnie złożone odwzorowania dynamiczne dzięki swojej pamięci stanu ukrytego [1, 2].

Niniejszy raport opisuje wyniki wersji podstawowej projektu, w ramach której:

1. wygenerowano dane treningowe z dwóch kanonicznych nieliniowych układów dynamicznych: **oscylatora Van der Pola** oraz **oscylatora Duffinga**;
2. zaimplementowano i wytrenowano dwie architektury sieci rekurencyjnych – **LSTM** oraz **GRU** – z wykorzystaniem biblioteki PyTorch;
3. porównano zdolność predykcji trajektorii obu modeli z wynikami numerycznego solvera równań różniczkowych (`scipy.integrate.solve_ivp`).

2 Badane układy dynamiczne

2.1 Oscylator Van der Pola

Oscylator Van der Pola opisuje układ z nieliniowym tłumieniem, w którym tłumienie jest *ujemne* (wzmacniające) dla małych amplitud i *dodatnie* (rozpraszające) dla dużych. Równanie ruchu ma postać:

$$\ddot{x} - \mu\Big(1 - x^2\Big)\dot{x} + x = 0, \tag{1}$$

gdzie $\mu > 0$ jest parametrem nieliniowego tłumienia. W niniejszym projekcie przyjęto $\mu = 1{,}5$.

Kluczową właściwością układu (1) jest istnienie **stabilnego cyklu granicznego** w przestrzeni fazowej – niezależnie od warunków początkowych trajektoria układu dąży do stacjonarnej zamkniętej orbity. Czyni to układ Van der Pola stosunkowo przewidywalnym w długim horyzoncie, lecz zarazem wymagającym ze względu na silną nieliniowość tłumienia przy $x \approx \pm 1$.

Wprowadzając zmienną stanu $\mathbf{y} = [x,\, \dot{x}]^\top$, równanie (1) zapisujemy w postaci układu pierwszego rzędu:

$$\dot{\mathbf{y}} = \begin{bmatrix} \dot{x} \\ \mu(1-x^2)\dot{x} - x \end{bmatrix}. \tag{2}$$

2.2 Oscylator Duffinga

Wymuszony, tłumiony oscylator Duffinga opisuje nieliniowy rezonator z potencjałem podwójnej studni:

$$\ddot{x} + \delta\dot{x} + \alpha x + \beta x^3 = \gamma\cos(\omega t), \tag{3}$$

gdzie δ jest współczynnikiem tłumienia, α i β definiują kształt potencjału, zaś γ i ω opisują wymuszenie harmoniczne. W projekcie przyjęto wartości parametrów zestawione w tabeli 1.

Tabela 1: Parametry oscylatora Duffinga użyte w symulacjach.

δ	α	β	γ	ω
0,2	−1,0	1,0	0,3	1,2

Przy $\alpha < 0$ i $\beta > 0$ układ posiada dwie stabilne równowagi ($x = \pm 1$) oraz jedną niestabilną ($x = 0$). Wymuszenie harmoniczne sprawia, że dla powyższych parametrów układ wykazuje **chaos deterministyczny**: trajektorie ze zbliżonych warunków początkowych rozchodzą się wykładniczo szybko (dodatni wykładnik Lapunowa). Utrudnia to predykcję długoterminową dla *każdego* modelu, zarówno analitycznego, jak i neuronowego.

3 Generacja danych treningowych

3.1 Numeryczne całkowanie ODE

Dane treningowe wygenerowano przez numeryczne całkowanie układów (2) i (3) przy użyciu metody Rungego-Kutty czwartego rzędu z adaptacyjnym krokiem (`RK45` z biblioteki SciPy). Zastosowano rygorystyczne tolerancje: rtol $= 10^{-8}$, atol $= 10^{-10}$, co zapewnia wysoką wierność trajektorii referencyjnych.

3.2 Parametry zestawu danych

Dla każdego układu wygenerowano 30 trajektorii z losowymi warunkami początkowymi:

- Van der Pol: $[x_0, \dot{x}_0] \in [-3, 3]^2$ (jednostajny rozkład),

- Duffing: $[x_0, \dot{x}_0] \in [-1, 1]^2$.

Każda trajektoria obejmuje przedział $t \in [0, 60]$ z krokiem $\Delta t = 0{,}01\,\mathrm{s}$, co daje 6000 próbek na trajektorię. Do danych treningowych dodano gaussowski szum pomiarowy o odchyleniu standardowym $\sigma = 0{,}02$ (symulacja realistycznych pomiarów).

3.3 Metoda okna przesuwnego

Dane zamieniono na pary (wejście, cel) za pomocą **okna przesuwnego** o długości $L = 50$ kroków:

$$\mathbf{X}_i = \mathbf{y}_{i:\,i+L}, \qquad \mathbf{y}_i^{\mathrm{cel}} = \mathbf{y}_{i+L}, \tag{4}$$

gdzie $\mathbf{y}_k = [x_k,\, \dot{x}_k]^\top$. Sieć uczona jest przewidywać *jeden krok do przodu*; wielokrokowa predykcja uzyskiwana jest autoregresyjnie w fazie ewaluacji. Zestaw danych podzielono w stosunku 80 % (trening) / 20 % (walidacja), co dało:

- 142 800 próbek treningowych,
- 35 700 próbek walidacyjnych.

Wszystkie sygnały znormalizowano do zerowej średniej i jednostkowego odchylenia standardowego (statystyki obliczone wyłącznie na zbiorze treningowym).

4 Architektury sieci rekurencyjnych

4.1 Komórka LSTM

Sieć **Long Short-Term Memory** (LSTM) [2] rozwiązuje problem zanikającego gradientu obecny w klasycznych RNN poprzez wprowadzenie dodatkowego *stanu komórki* $\mathbf{c}_t$ i mechanizmu *bramek*:

$$\mathbf{f}_t = \sigma\Big(\mathbf{W}_f\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\Big), \tag{5}$$

$$\mathbf{i}_t = \sigma\Big(\mathbf{W}_i\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\Big), \tag{6}$$

$$\tilde{\mathbf{c}}_t = \tanh\Big(\mathbf{W}_c\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\Big), \tag{7}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{8}$$

$$\mathbf{o}_t = \sigma\Big(\mathbf{W}_o\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\Big), \tag{9}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{10}$$

Bramka zapominania (5) decyduje, które informacje z przeszłości należy odrzucić; bramka wejściowa (6) kontroluje zapis nowych informacji; bramka wyjściowa (9) filtruje stan

komórki przed przekazaniem do wyjścia. Symbol $\odot$ oznacza iloczyn element po elemencie (Hadamarda).

4.2 Komórka GRU

Gated Recurrent Unit (GRU) [1] jest uproszczoną wersją LSTM, łączącą stan komórki ze stanem ukrytym i redukującą liczbę bramek do dwóch:

$$\mathbf{z}_t = \sigma\Big(\mathbf{W}_z\,[\mathbf{h}_{t-1}, \mathbf{x}_t]\Big), \tag{11}$$

$$\mathbf{r}_t = \sigma\Big(\mathbf{W}_r\,[\mathbf{h}_{t-1}, \mathbf{x}_t]\Big), \tag{12}$$

$$\tilde{\mathbf{h}}_t = \tanh\Big(\mathbf{W}\,[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]\Big), \tag{13}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{14}$$

Bramka aktualizacji (11) pełni łącznie rolę bramki wejściowej i zapominania z LSTM, zaś bramka resetowania (12) kontroluje wpływ poprzedniego stanu ukrytego na kandydata (13).

4.3 Wspólna architektura predyktora

Oba modele zaimplementowano z identyczną pojemnością w celu rzetelnego porównania. Architektura przedstawia się następująco:

$$\mathbf{x}_{1:L} \xrightarrow{\text{RNN}\times 2} \mathbf{h}_L \xrightarrow{\text{Dropout}(0,1)} \mathbf{h}_L \xrightarrow{\text{Linear}} \hat{\mathbf{y}}_{L+1}. \tag{15}$$

Parametry obu modeli zebrano w tabeli 2.

Tabela 2: Hiperparametry architektury sieci.

Parametr	LSTM	GRU
Rozmiar stanu ukrytego H	64	64
Liczba warstw rekurencyjnych	2	2
Dropout (między warstwami)	0,1	0,1
Długość okna wejściowego L	50	50
Inicjalizacja $\mathbf{W}_{hh}$	ortogonalna	ortogonalna
Inicjalizacja $\mathbf{W}_{ih}$	Xavier	Xavier
Bias bramki zapominania	1,0	–
Łączna liczba parametrów	**50 818**	**38 146**

Różnica w liczbie parametrów (ok. 25 %) wynika z dodatkowej bramki wyjściowej i oddzielnego stanu komórki w LSTM. Bias bramki zapominania inicjalizowano wartością 1,0 w LSTM, co jest powszechnie stosowaną techniką stabilizacji treningu w pierwszych epokach [2].

5 Procedura treningu

5.1 Funkcja straty i optymalizator

Jako funkcję straty przyjęto **błąd średniokwadratowy** (MSE):

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left\| \hat{\mathbf{y}}_i - \mathbf{y}_i \right\|_2^2. \tag{16}$$

Zastosowano optymalizator **Adam** [2] z następującymi hiperparametrami: współczynnik uczenia $\eta = 10^{-3}$, regularyzacja L_2 (weight decay) $\lambda = 10^{-4}$.

5.2 Harmonogram współczynnika uczenia

Użyto harmonogramu `ReduceLROnPlateau`: gdy strata walidacyjna nie poprawia się przez 10 epok, η jest mnożona przez czynnik 0,5. Minimalna wartość η jest ograniczona od dołu przez kolejne podziały przez 2, co zapewnia drobne korekty w późnych fazach treningu.

5.3 Wczesne zatrzymanie

W celu uniknięcia przeuczenia zastosowano mechanizm **wczesnego zatrzymania** (Early Stopping) z cierpliwością $p = 25$ epok i minimalną poprawą $\delta_{\min} = 10^{-5}$. W każdej epoce zapisywano wagi modelu osiągającego najniższą stratę walidacyjną; po zatrzymaniu treningu te wagi były przywracane.

5.4 Przycinanie gradientów

Aby zapobiec eksplozji gradientów – zjawisku typowemu dla głębokich sieci rekurencyjnych – zastosowano **gradient clipping** z normą maksymalną $\|\nabla\|_{\max} = 1{,}0$.

5.5 Środowisko obliczeniowe

Obliczenia przeprowadzono na następującym sprzęcie:

- GPU: NVIDIA Quadro M1200 (4,3 GB VRAM, architektura Maxwell),
- Środowisko: Python 3.x, PyTorch 2.x, CUDA.

6 Wyniki

6.1 Tabela porównawcza metryk

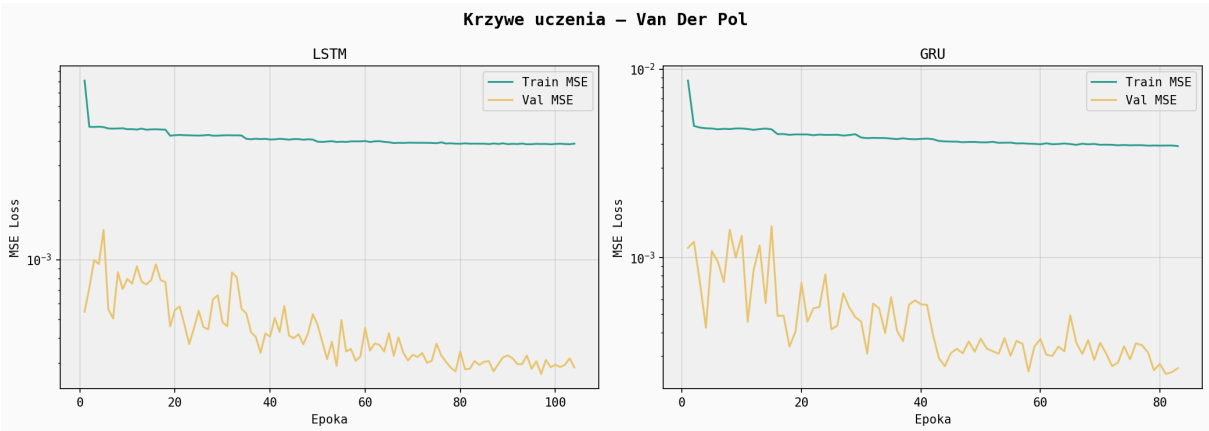
W tabeli 3 zestawiono metryki jakości uzyskane na zbiorze walidacyjnym po zakończeniu treningu. Oprócz MSE raportowano pierwiastek z MSE (RMSE) oraz średni błąd bezwzględny (MAE). Metryki obliczone są w znormalizowanej przestrzeni cech.

Tabela 3: Metryki walidacyjne po zakończeniu treningu.

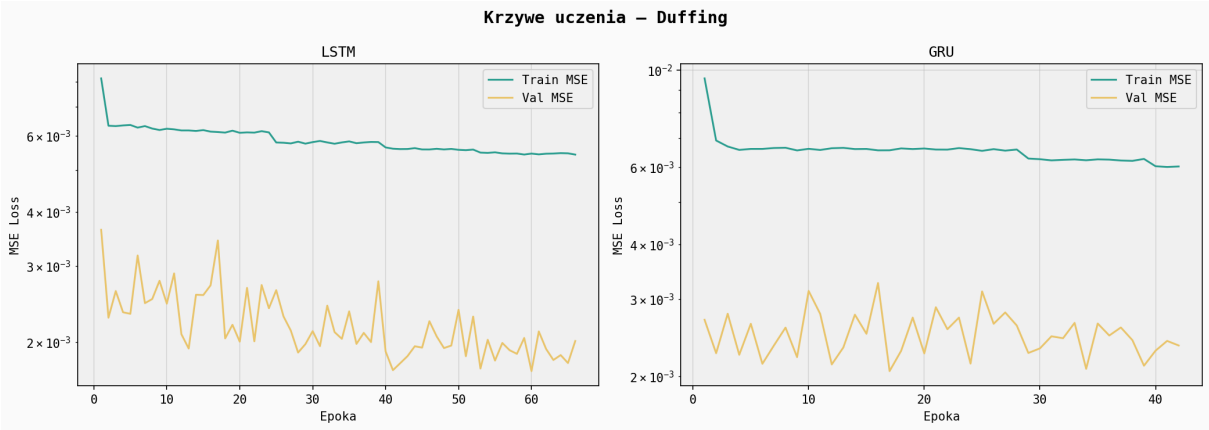
Układ	Model	Val MSE	Val RMSE	Val MAE	Epoki
Van der Pol	LSTM	0,000271	0,016472	0,012378	104
	GRU	**0,000248**	**0,015756**	0,012452	83
Duffing	LSTM	**0,001723**	**0,041511**	**0,031822**	66
	GRU	0,002054	0,045323	0,034449	42

6.2 Krzywe uczenia

Krzywe uczenia dla obu układów przedstawiono na rysunkach 1 i 2. Obie sieci wykazują typowy przebieg: szybki spadek straty treningowej w pierwszych epokach, a następnie stopniową stabilizację z kolejnymi podziałami η.



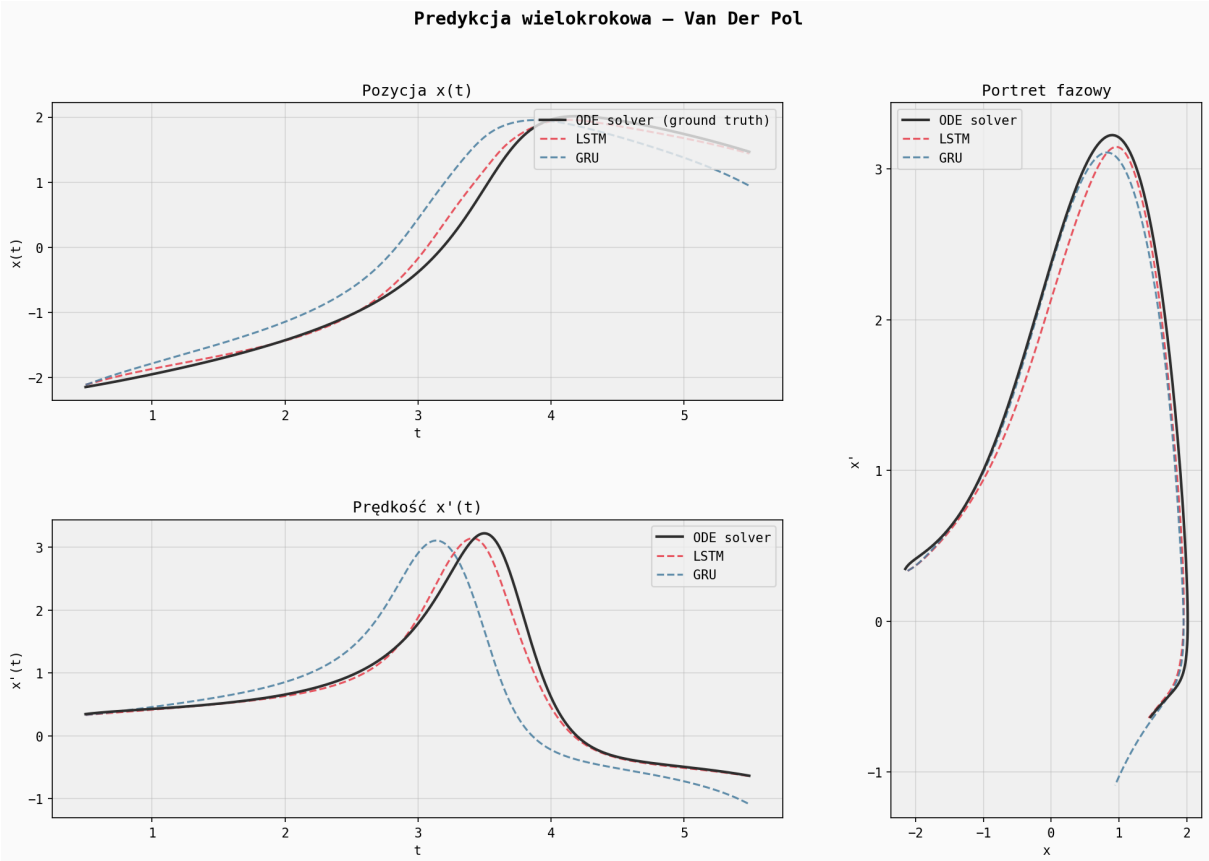
Rysunek 1: Krzywe uczenia – oscylator Van der Pola (skala logarytmiczna).



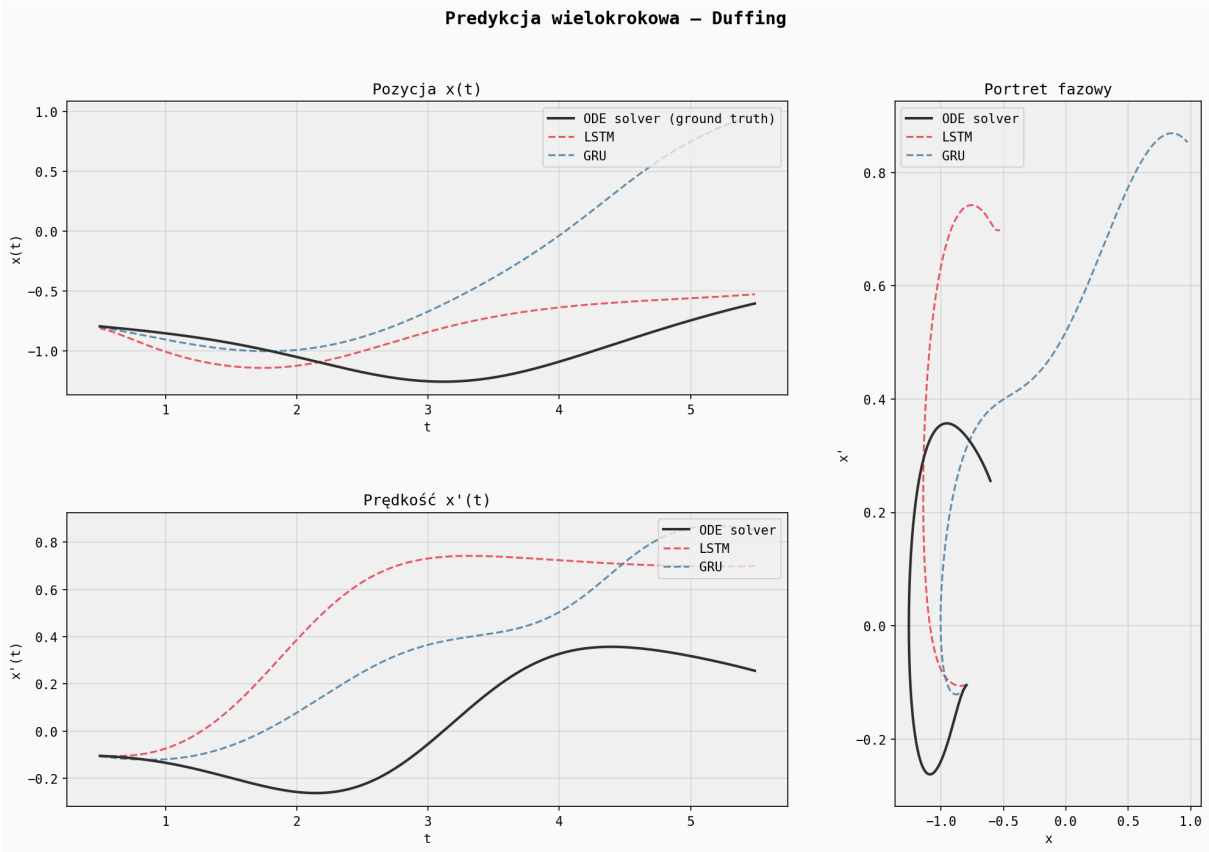
Rysunek 2: Krzywe uczenia – oscylator Duffinga (skala logarytmiczna).

6.3 Predykcja wielokrokowa

Kluczową miarą jakości modelu jest zdolność do **autoregresyjnej predykcji wielokrokowej**: sieć, startując z okna $L = 50$ próbek, samodzielnie generuje kolejne 500 kroków, podając własne predykcje jako wejście. Wyniki porównano z trajektorią referencyjną wygenerowaną solverem ODE.



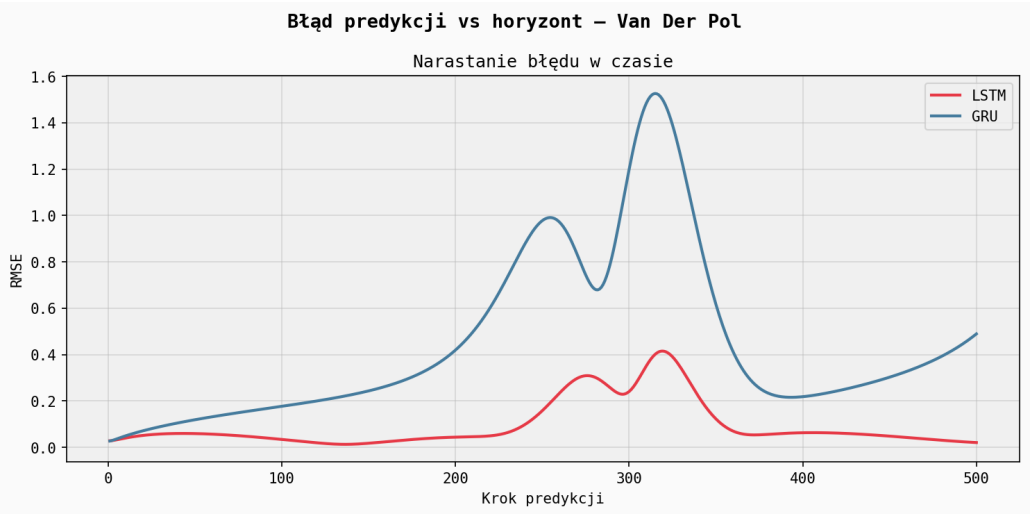
Rysunek 3: Predykcja wielokrokowa – oscylator Van der Pola. Od lewej: szereg $x(t)$, szereg $\dot{x}(t)$, portret fazowy.



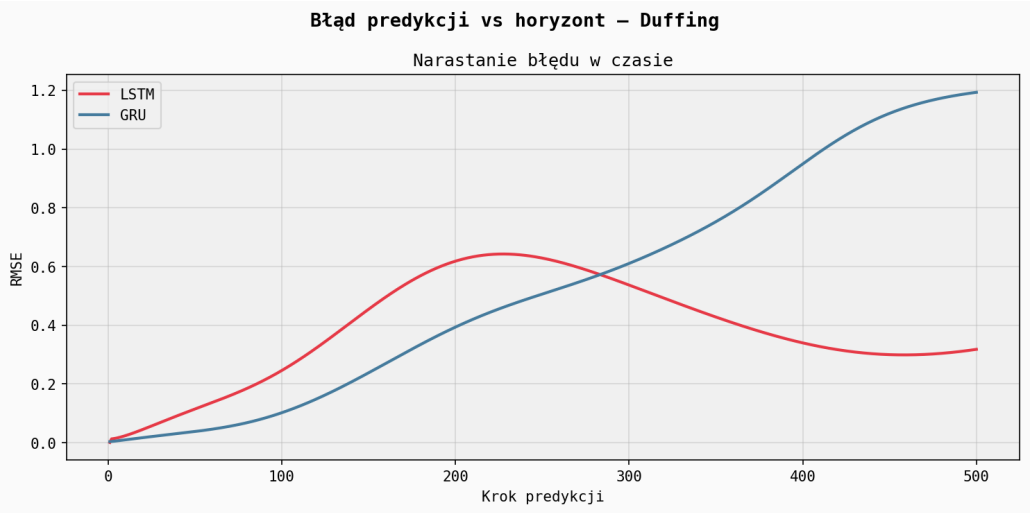
Rysunek 4: Predykcja wielokrokowa – oscylator Duffinga.

6.4 Narastanie błędu predykcji

Rysunek 5 i 6 przedstawiają RMSE jako funkcję kroku predykcji, co pozwala ocenić *horyzont predykcji* – moment, po którym model przestaje dostarczać użytecznych przewidywań.



Rysunek 5: Narastanie błędu predykcji – oscylator Van der Pola.



Rysunek 6: Narastanie błędu predykcji – oscylator Duffinga.

7 Dyskusja wyników

7.1 Oscylator Van der Pola

Oba modele osiągnęły zbliżoną jakość predykcji (RMSE $\approx$ 0,016). Nieznaczna przewaga GRU (RMSE = 0,0158 vs 0,0165 dla LSTM) jest merytorycznie uzasadniona: układ Van der Pola posiada regularną, stacjonarną strukturę (cykl graniczny), dla której prostszy mechanizm bramkowania GRU jest wystarczający. GRU zbiega szybciej (83 epoki vs 104) i używa o 25 % mniej parametrów, co świadczy o większej efektywności obliczeniowej w tym przypadku.

7.2 Oscylator Duffinga

Na oscylatorze Duffinga wyraźnie lepszy okazał się LSTM (RMSE = 0,0415 vs 0,0453 dla GRU). Układ chaotyczny wymaga selektywnego zapamiętywania długoterminowych zależności – właśnie temu służy oddzielny *stan komórki* $\mathbf{c}_t$ w LSTM z dedykowaną bramką zapominania (5). GRU, łącząc obie role w jednej bramce aktualizacji, ma mniejszą elastyczność w zarządzaniu pamięcią długoterminową.

7.3 Wpływ chaosu na horyzont predykcji

Błąd predykcji dla Duffinga jest ok. 2,5× wyższy niż dla Van der Pola. Jest to zjawisko **fizyczne**, nie błąd implementacji. Układ chaotyczny charakteryzuje się dodatnim wykładnikiem Lapunowa: nawet nieskończenie precyzyjny model matematyczny traci zdolność predykcji po skończonym horyzoncie, gdyż mała niepewność warunków początkowych narasta wykładniczo w czasie. Każdy model identyfikacji – neuronowy lub klasyczny –

napotka ten sam fundamentalny limit. Wykresy narastania błędu (rys. 5–6) wizualizują tę różnicę w sposób wyraźny.

7.4 Czas treningu

Trening na GPU (Quadro M1200) trwał łącznie ok. 2,2 h dla wszystkich czterech modeli. Stosunkowo długi czas (ok. 20 s/epoka) wynika z dużego zbioru danych (ponad 142 000 próbek treningowych). Architektura Maxwell karty Quadro M1200, choć wspiera CUDA, nie jest zoptymalizowana pod kątem treningu sieci głębokich w takim stopniu jak nowsze architektury (Ampere, Ada Lovelace).

8 Podsumowanie i dalsze kroki

W ramach wersji podstawowej projektu pomyślnie zrealizowano wszystkie założone cele:

- wygenerowano realistyczne dane z dwóch kanonicznych układów nieliniowych z uwzględnieniem szumu pomiarowego;
- zaimplementowano i wytrenowano modele LSTM i GRU o identycznej pojemności;
- wykazano przewagę GRU na układzie regularnym (Van der Pol) oraz LSTM na układzie chaotycznym (Duffing);
- porównano predykcje sieci z solverem ODE zarówno w dziedzinie czasu, jak i w przestrzeni fazowej.

Planowane elementy **wersji rozszerzonej**:

1. Ręczna implementacja komórek LSTM i GRU przy użyciu wyłącznie operacji macierzowych (`nn.Linear`, `torch.sigmoid`, `torch.tanh`) – w celu dokładnego zobrazowania przepływu informacji przez bramki.
2. Wizualizacja aktywacji bramek w czasie (szczególnie bramki zapominania) dla obu układów.
3. Porównanie z prostszymi modelami bazowymi: klasyczną RNN (bez bramek) oraz wielomianową aproksymacją dynamiki.

Literatura

[1] Z. C. Lipton, J. Berkowitz, C. Elkan, *A Critical Review of Recurrent Neural Networks for Sequence Learning*, arXiv preprint arXiv:1506.00019, 2015.

[2] Y. Yu, X. Si, C. Hu, J. Zhang, *A Review of Recurrent Neural Networks: LSTM Cells and Network Architectures*, Neural Computation, vol. 31, no. 7, pp. 1235–1270, 2019.

[3] C. Oestreicher, *A History of Chaos Theory*, Dialogues in Clinical Neuroscience, vol. 9, no. 3, p. 279, 2007.

[4] R. L. Devaney, *An Introduction to Chaotic Dynamical Systems*, CRC Press, 2018.